Braeden Venne

8561 Midterm

Part A - Architecture Design



Braeden Venne

Architecture Overview: The system uses a client-server setup with simple text-based application protocol running over TCP sockets. The main components of this architecture are client.py and server.py, which communicate with each other using a COMMAND|CONTENT message format. The security tools port_scanner.py, http_security_checker.py, and send_message.py run locally on the client side and are accessed through the main security tools menu UI, while the server handles connection management, authentication, locking down unauthorized access, and running the analyze_pcap.py module.

1. **Component Responsibilities:**
    - **TCP Client (client.py):** Handles the main user interface and security assessments menu. It opens the TCP connection to the server, grabs user credentials using getpass so passwords are not displayed on screen, and displays the security assessment tools menu which a user can input an option on which tool they want to run.
    - **Port Scanner (port_scanner.py):** A client-side module that resolves target hostnames to IPs, validates the user inputted port numbers fall between 1 and 65535, runs scans using python-nmap, maps open ports to their known services, and prints the formatted results in a table,
    - **HTTP Security Checker (http_security_checker.py):** A client-side module that sends HTTP GET requests to server endpoints (/, /health, /redirect, /missing) using requests. It checks for missing security headers, checks cookie flags, catches connection errors, and saves findings to CSV files.
    - **Message Application (send_message.py):** A client-side messaging module that runs an active chat loop with the server. It formats user inputs with the MSG| header, ensures messages stay under 200 characters, and returns to the main menu when the user types “EXIT”.
    - **TCP Server (server.py):** Listens on 127.0.0.1:12345 for client connections. It tracks session states, splits incoming commands on the pipe delimiter, hashes passwords with SHA-256 to check against the user dictionary, verifies OTP codes using pyotp, locks out clients after three failed login attempts, and logs events to security_events.log.
    - **PCAP Analyzer (analyze_pcap.py):** A server-side packet analysis module using Scapy upon receiving a PCAP| command. It uses Scapy to parse the provided PCAP file, counts TCP and UDP packets, and flags any source Ips sending more than 20 packets in a 5-second window.
2. **Information Flows:**
    - **Connection:** The client opens a standard TCP socket to 127.0.0.1:12345.
    - **Authentication:** The client sends “AUTH|username|password” to the server. The server hashes the password and checks it against its stored hash. If the hash matches, it replies with OTP_REQUIRED, proceeding to the next stage. The user then enters their OTP code on the terminal which is sent to the server as “OTP|code”. The server then verifies the code using pyotp.
    - **Security Assessment Tool Menu Access:** Once the server verifies the credentials and OTP it returns an ACCESS_GRANTED message to the client, the client then uses that to unlock the menu options

- **Tool Execution:** The menu will display five options that the user can select from. Depending on what the client chooses it will either run one of the local tools or send a command over the socket for the server to process and reply to,

3. **Rationale for the Stateful Interaction Model:**
   - The reason for having a stateful interaction menu is because TCP holds an open connection, the server can easily track the client's current state across multiple steps instead of treating each request independently. It also remembers whether the socket has passed both the password and OTP verification, which allows the server to enforce strict access checks.

Part B - Adversarial Threat Model

Threat	Asset	Attack Vector	Impact	Mitigation	Residual Risk
Brute-Force Attacks	User authentication system (usernames, passwords, and OTP).	Repeated automated login attempts sent over the TCP socket.	Account can be compromised or unauthorized access.	A failed login attempt lockout threshold has been set.	Distributed attacks across multiple sessions.
Credential Sniffing	Plaintext credentials in transit.	An attacker can use a passive packet capture (i.e. Wireshark) on the TCP channel.	Credentials can be stolen and sessions can be hijacked.	Time-based MFA with pyotp limits the token lifespan, reducing the risk of attackers logging in even if they get the credentials.	Passwords and messages can still be read by a third party watching the network traffic.
DDoS Attack	Server socket connection queue, CPU, and memory.	Flooding the server with simultaneous connection requests or malformed loops holding sockets open.	Server crashes or becomes unresponsive to legitimate user connections.	The application currently only has a single connection limit.	The single client architecture lacks high availability flood mitigation.
Unauthorized Port Scanning	Local port scanner module (nmap).	Running scans against an external website or inputting the wrong IP address into the prompt.	Network disruption or triggering IDS/IPS alerts.	Input validation restricting targets to 127.0.0.1 (localhost).	User error during manual IP address input.
Unauthorized Command Execution / State Bypass	Application modules (MSG, PCAP, scanning tools) and	An unauthenticated client sending commands like	Unauthorized users can access restricted tools	Server-side stateful authorization checks that	Session state tracking issues if connections drop unexpectedly.

	backend application logic.	MSG or PCAP out of order without completing login.	or send commands without logging in.	block commands and return error messages	

Part C - Protocol Design

Specify the TCP application-layer protocol: message structure and command format; authentication and MFA sequence; authorization rules for protected commands; malformed request handling; error responses; session termination; and replay/security limitations. Clearly distinguish protections implemented from those proposed for future deployment.

1. Message Structure and Command Format
    - The application uses a custom, text-based application-layer protocol running over a standard TCP socket (SOCK_STREAM)
        - Messages use a strict format of “COMMAND|CONTENT” separating the command and content of the message by the pipeline (|) character.
        - Commands that may require multiple parameters, such as AUTH|username|password, split them with additional pipes.
        - Supported commands include AUTH, OTP, MSG, PCAP, and EXIT.

2. User Authentication and MFA Sequence
    - Authentication is handled in a multiple step challenge-response flow after the user establishes a connection the server.
        - **Password Verification Step**: The client is prompted to input their username and password; this is then sent to the server as “AUTH|username|password”. The server then hashes the inputted password with SHA-256 and compares it against the stored hash in the mock user database (users). If the incorrect password is entered the server returns an “ACCESS_DENIED” message and increments the failure counter.
            - If three failed attempts are reached, the server ends the connection with the client and returns a “ACCESS_DENIED|Too many failed attempts. Authentication blocked” message. The socket is then closed.
        - **MFA Step:** If the password passes the verification step, the server replies with “OTP_REQUIRED”. The client then prompts the user for an OTP code and gets sent back to the server as “OTP|code”.
        - **OTP Verification Step:** The server validates the inputted OTP code using the pyotp library. If the code is valid, it responds with “ACCESS_GRANTED” which unlocks the security applications menu for the user. If the code is invalid, it returns an “ACCESS_DENIED” message and increments the failure counter.
            - If three failed attempts are reached, the server ends the connection with the client and returns a “ACCESS_DENIED|Too many failed

attempts. Authentication blocked" message. The socket is then closed.

3. Authorization Rules for Protected Commands
    - The server enforces state-based authorization before the user can complete specific actions. For example, if an unauthenticated client tries to run the PCAP analysis by sending the "PCAP|" command or send a message to the server with the "MSG|" command, the server immediately rejects it with "ERROR|User authentication is required".

4. Malformed Request Handling and Error Responses
    - **Format Errors:** Every incoming message from the user is checked for the pipe delimiter (|). If the pipe is missing, the server will return an "ERROR|Invalid command format" message to the user.
    - **Empty Inputs:** If the user sends an empty username, password, OTP, or chat message, the server catches them and will return specific error messages such as "ERROR|Username required" or "ERROR|Message cannot be empty".
    - **Message Length Limit:** The server enforces a strict limit of 200 characters for the "MSG" command. If this limit is exceeded the server will return an "ERROR|Message too long" message to the client.

5. Session Termination
    - There are currently three ways that a session can be terminated by the server.
        - **Normal Exit:** The user selects option 5 from the menu, which sends an "EXIT|" command to the server. The server will then reply with an "OK|Goodbye" message, closing the connection on both sides gracefully.
        - **Unexpected Disconnection:** If a client abruptly closes the window or drops the connection, the server catches the "ConnectionResetError" or empty socket data, prints a disconnection notice, then cleans up the sockets.
        - **Exceeding Failed Login Attempt Threshold:** If a user exceeds the strict three failed attempt threshold (MAX_FAILED_ATTEMPTS = 3) on the login stage the server will return a "ACCESS_DENIED|Too many failed attempts. Authentication blocked" message and close the socket.

6. Replay and Security Limitations (Implemented vs Proposed)
    - What is Currently Implemented:
        - Time-based one-time passwords with pyotp prevent attackers from reusing captured OTP codes outside of their short 30 second validity window
        - SHA-256 password hashing prevents raw passwords from being stored in plaintext on the server backend

- Proposed for Future Deployment:
    - Since this application runs over plain, unencrypted TCP sockets, all usernames, passwords, OTP codes, and commands travel across the network in plaintext. In the future TLS/SSL using Python's ssl library should be implemented so that the entire TCP channel is encrypted, preventing packet sniffing.

Part D - Component Design

1. **TCP Client (client.py)**
    - **Inputs:** Usernames, passwords (via secure prompt), OTP codes, target hostnames, port ranges, chat messages, and menu selections which include the following five options:
        1. Run Local TCP Port Scanner
        2. Run HTTP Website Security Checker
        3. Run Offline PCAP Analysis
        4. Run Message Application
        5. Exit Application
    - **Outputs:** Interactive terminal menus, authentication challenge prompts, formatted scan outputs, server responses.
    - **Expected Behavior:** Establishes a TCP socket connection to 127.0.0.1:12345, runs the interactive authentication loop (sending AUTH and OTP commands). Renders the Security Assessment Tools menu upon successful authentication and verification, translates user choices into framed network commands (PCAP, EXIT, MSG).
    - **Failure Conditions:** Connection errors or socket drops are caught, prompting clean exit messages if the server goes offline. Invalid menu choices, failed authentication attempts, and empty user inputs trigger respective re-loops or blockages.
    - **Security Controls:** Passwords are entered securely with getpass to prevent terminal echoing, stateful authentication is tracked.

2. **TCP Server (server.py) – Also handles the core MFA module**
    - **Inputs:** Inbound client socket connections, raw command strings transmitted over the TCP stream, mock database records (users), and server configuration parameters (MAX_FAILED_ATTEMPTS = 3).
    - **Outputs:** State progression signals (OTP_REQUIRED, ACCESS_GRANTED, ACCESS_DENIED), command execution results, error strings, and terminal logging notices.
    - **Expected Behavior:** Binds to 127.0.0.1:12345, listens for a client connection, parses incoming messages using pipe delimiters (COMMAND|CONTENT), maintains sessions state by tracking whether a client is authenticated and counting failed authentication attempts, and routes authorized requests to their respective application modules.
    - **Failure Conditions:** Unexpected client disconnections raise a ConnectionResetError or empty socket reads, which the server catches to close the socket and reset state

cleanly. Malformed strings missing a pipe delimiter return an immediate “ERROR|Invalid command format” response. Client exceeding the preset failed attempt threshold.

- **Security Controls:** Delimiter checking, stateful authorization gating, SHA-256 password hashing, pyotp token verification, failure lockout thresholds, and security event logging that captures failed logins, lockouts, and unauthorized command attempts.

3. **Port Scanner (port_scanner.py)**
    - **Inputs:** Target host string (i.e. localhost or 127.0.0.1) and start/end port integers (ranging from 1 to 65535).
    - **Outputs:** Formatted terminal text/tables detailing every port in the tested range alongside its current state (i.e. open or closed) and its mapped service name or “unknown” if applicable.
    - **Expected Behavior:** Resolves the target hostname to an IP address using socket.gethostbyname(), executes a TCP port scan using the python-nmap library, and loops through the full user-defined range to display active services and inactive ports.
    - **Failure Conditions:** Invalid hostnames raise socket.gaierror exceptions caught by retry logic. Out of order port bounds (end port is smaller than start port) or non-integer entries trigger validation re-prompt loops.
    - **Security Controls:** Input validation boundaries ensuring port values stay strictly within TCP limits (1-65535).

4. **Message Application Module (send_message.py)**
    - **Inputs:** Chat message strings typed by the user into the active terminal session.
    - **Outputs:** Formatted protocol messages “MSG|content” sent over the active socket and server acknowledgment responses.
    - **Expected Behavior:** Operates an active communication loop once the user enters the chat application. Inputted chat messages have the “MSG|” tag attached to them and are then transmitted to the server. Users can type “EXIT” to return to the main menu.
    - **Failure Conditions:** Blank message strings or messages exceeding the string content length threshold are caught and rejected by local and server-side validation rules
    - **Security Controls:** Strict 200-character length limit is enforced to protect backend buffer allocations.

5. **HTTP Security Checker (http_security_checker.py)**
    - **Inputs:** Target base URL (http://127.0.0.1:8765) and an array of specific test path endpoints (/, /health, /redirect, /missing)
    - **Outputs:** Real-time console logs, vulnerability observation summaries, and two exported CSV report files (http_scan_results.csv and vulnerability_findings.csv).

- **Expected Behavior:** Issues HTTP GET requests using the requests library, inspects response headers for missing security controls or version disclosure signatures, exports findings to CSV files and client terminal.
- **Failure Conditions:** If the target local test service is offline, requests.RequestException catches connection failures, preventing application crashes and allowing the client to continue running.
- **Security Controls:** Uses try-except blocks so the script doesn't crash if the target website goes offline, and checks findings before writing them to make sure it doesn't add duplicate rows in the exported CSV files.

6. **PCAP Analyzer (analyze_pcap.py)**
    - **Inputs:** File path pointing to the target packet capture file (FSCT8561_Operation_Sentinel_Traffic.pcap)
    - **Outputs:** Terminal summary displaying the total TCP and UDP packet counts, along with a list of flagged suspicious source IPs that exceed traffic thresholds.
    - **Expected Behavior:** Parses the PCAP file using Scapy's rdpcap() function, counts the TCP and UDP packets, and flags any IP address that send more than 20 packets within a 5-second window as suspicious.
    - **Failure Conditions:** Missing packet capture file or incorrect spelling of the path, file exceptions are caught and reported back cleanly.
    - **Security Controls:** Exception handling around file operations to ensure backend script stability.

7. **Sensitive Information Exclusion and Justification**
    - Plaintext passwords, account password hash strings, and raw TOTP secret keys were excluded from being collected and logged.
    - These were excluded due to their sensitive nature, I wanted to ensure that even if an unauthorized user gains read access to log outputs or captures diagnostic data, they would not be able to recover raw credentials or forge valid MFA tokens.

Part E – Full Integrated Implementation

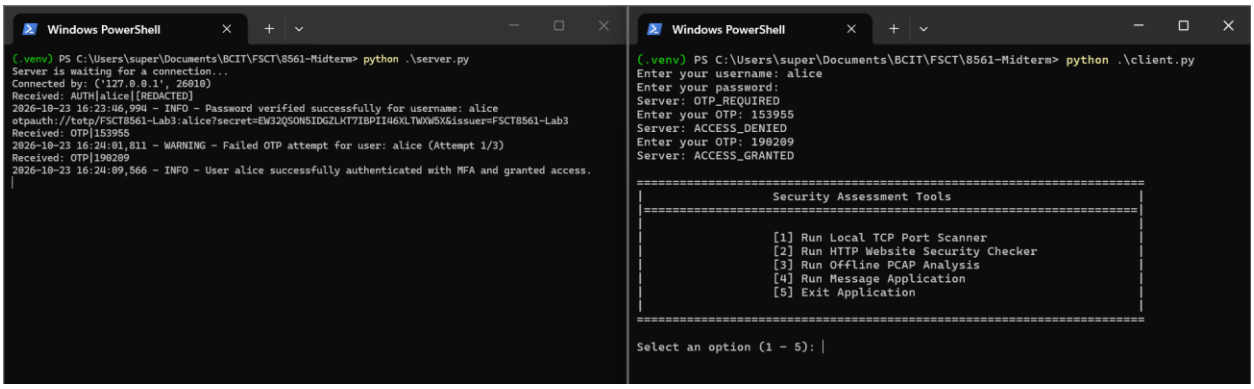
The code for my fully integrated implementation is on the following GitHub Repository link: https://github.com/BraedenVenne/8561-Midterm

There is also a README.md file on the GitHub Repository explaining step by step on how to run the application.

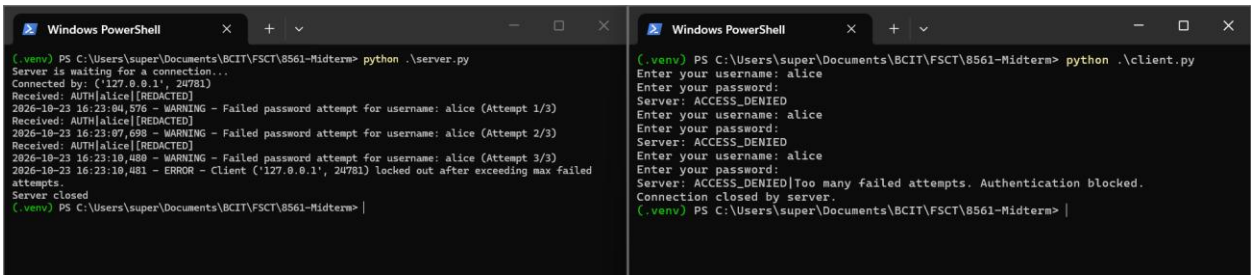
Part F – Comprehensive Testing

For this stage I have laid out the following tests:

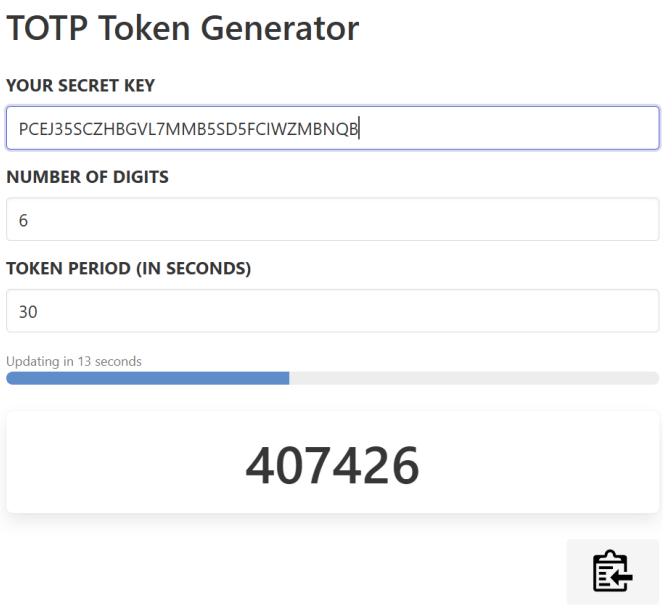
1. Successful Authentication
   - **Test Description:** Authenticating a valid user (alice) with correct credentials and a valid TOTP code.
   - **Expected Result:** The server validated the password, requests an OTP (OTP_REQUIRED), receives the user inputted OTP and validates it via pyotp, returns ACCESS_GRANTED, and unlocks the main security tools menu.
   - **Actual Result:** Server successfully verified the credentials, accepted the valid OTP, returned ACCESS_GRANTED, and displayed the security tools menu
   - **Status:** Passed
   - **Evidence:**



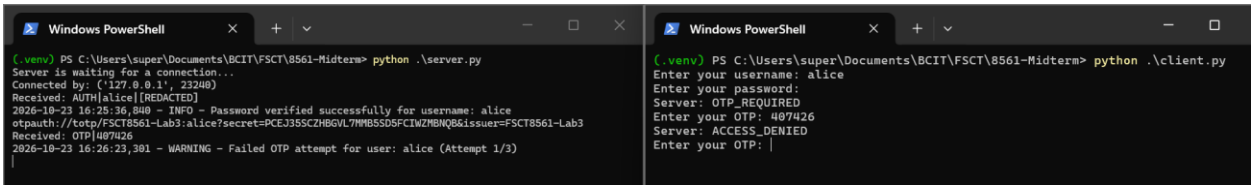
2. Authentication Lockout After Exceeding Failed Attempt Threshold
   - **Test Description:** Exceeding the maximum allowed failed login attempts (MAX_FAILED_ATTEMPTS = 3).
   - **Expected Results:** Server tracks failed password or OTP attempts, increments the counter, and upon reaching the 3rd failure, terminates the session with an "ACCESS_DENIED|Too many failed attempts. Authentication blocked" message and closes the socket.
   - **Actual Result:** After three incorrect login attempts, the server blocked the connection, logged the security event, and closed the socket cleanly.
   - **Status:** Passed
   - **Evidence:**



3. Sending Expired OTP
    - **Test Description:** Sending an expired/previous TOTP code to the server.
    - **Expected Results:** The server rejects the TOTP code, logs a failed OTP attempt, returned ACCESS_DENIED, and re-prompted the client for a valid token.
    - **Actual Result:** The server rejected the expired code, logged a failed OTP attempt, returned ACCESS_DENIED, and re-prompted the client for a valid token
    - **Status:** Passed
    - **Evidence:**

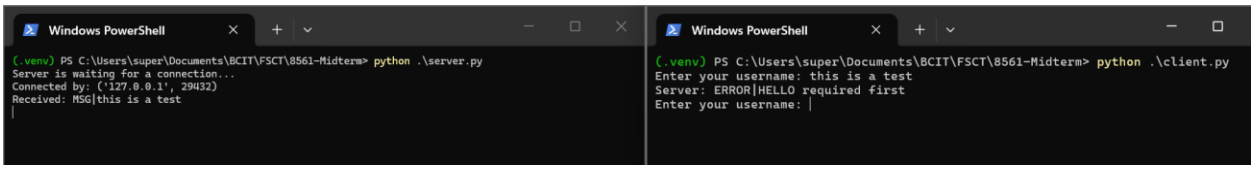


    - Waited for secret to expire

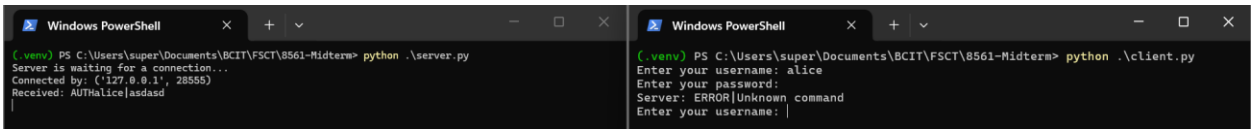


    - Didn't accept it
4. Sending a Command Before Authentication Stage
    - **Test Description:** Attempting to send a command before completing the required authentication and MFA sequence.
    - **Expected Results:** The server catches the out of order request, denies access, and returns an "ERROR|HELLO required first" to maintain the proper command sequence.
    - **Actual Result:** The server caught the out of order request, denied access, and returns an "ERROR|HELLO required first" to maintain the proper command sequence.

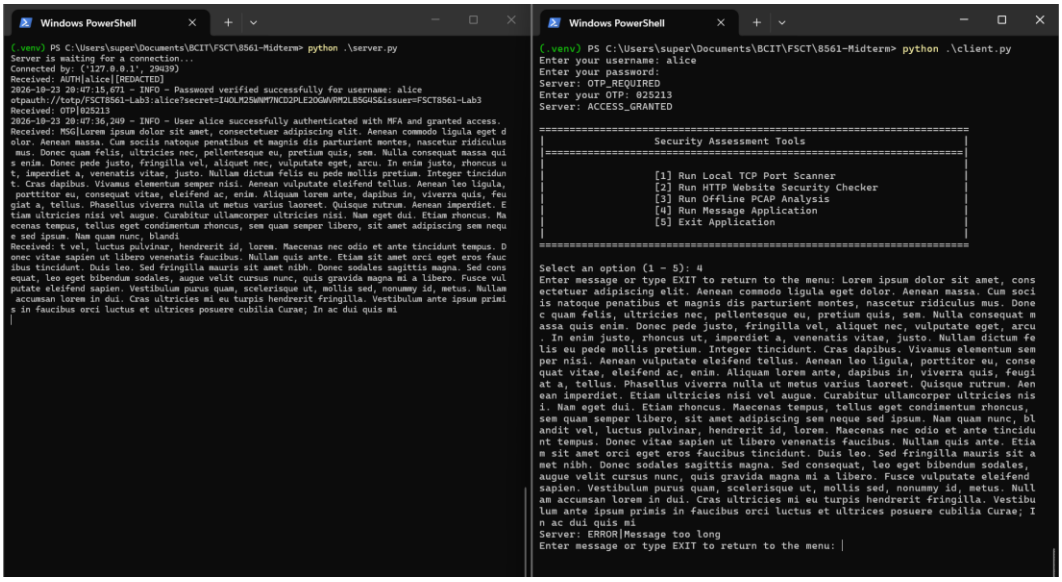
- **Status:** Passed
- **Evidence:**



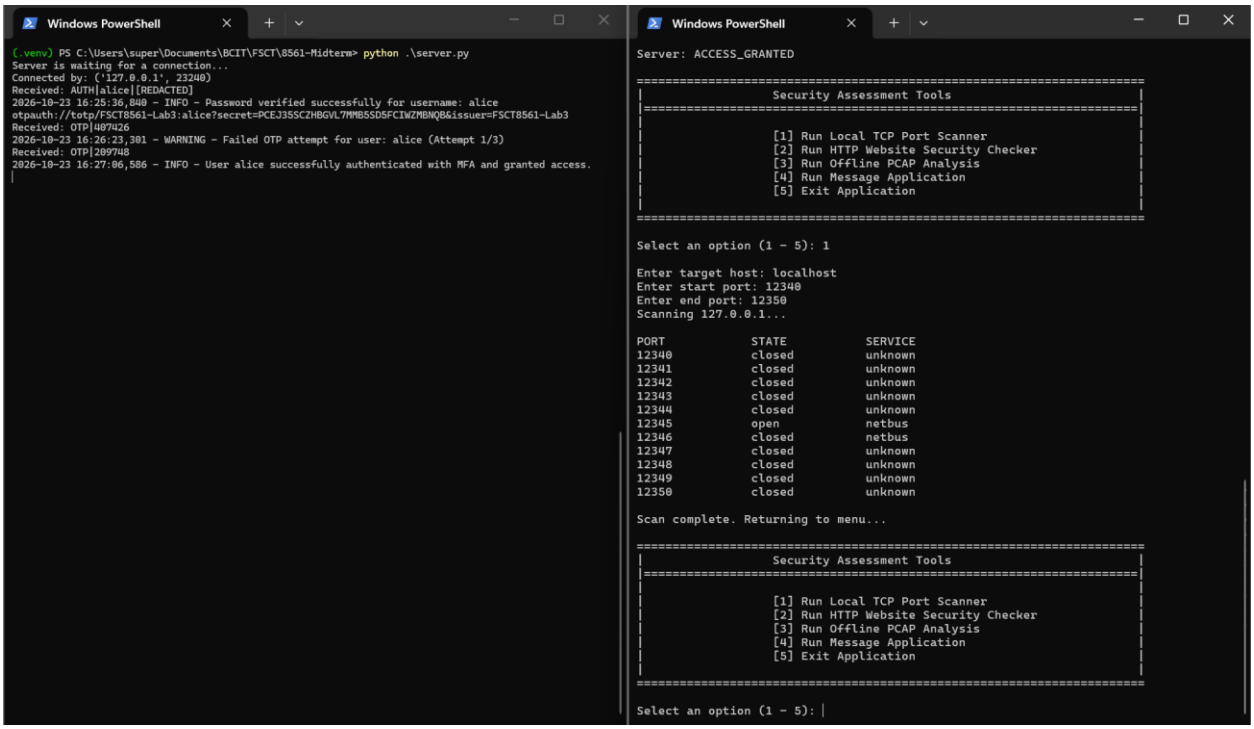
5. Input With Incorrect Structure
   - **Test Description:** Sending a command string that doesn't follow the required COMMAND|CONTENT structure.
   - **Expected Results:** The server catches the malformed format and responds with an "ERROR|Unknown command" message.
   - **Actual Result:** The server caught the malformed format and responded with an "ERROR|Unknown command" message.
   - **Status:** Passed
   - **Evidence:**



6. Exceeding Input Length
   - **Test Description:** Sending a chat message (MSG) that exceeds the maximum string length limit of 200 characters.
   - **Expected Results:** The server catches the oversized message and rejects it with an "ERROR|Message too long" to protect against buffer overflows.
   - **Actual Result:** The server caught the oversized message and rejected it with an "ERROR|Message too long" message and kept the session stable.
   - **Status:** Passed
   - **Evidence:**

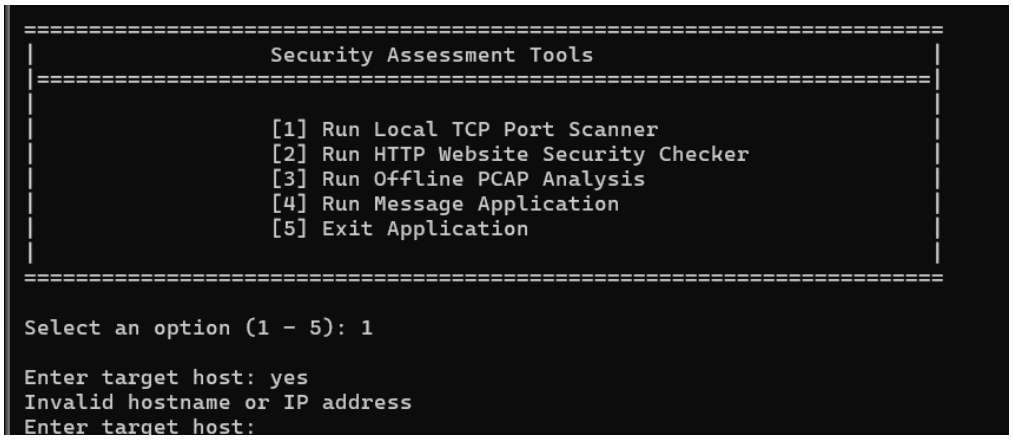


7. Running the Port Scanner Module
    - **Test Description:** Running the local TCP port scanner module from the security tools menu, providing a valid host (localhost) and port range (12340-12350).
    - **Expected Results:** The scanner resolves the hostname to an IP address, executes a TCP port scan using python-nmap. And outputs a formatted table listing each port alongside its state (open or closed) and service mapping.
    - **Actual Result:** The scanner successfully queried the target range, identified open ports (12345 running netbus) and closed ports, and returned to the security tools menu once finished.
    - **Status:** Passed
    - **Evidence:**



8. Inputting an invalid host on the Port Scanner Module
    - **Test Description:** Running the TCP port scanner module from the authenticated menu and providing an invalid host.
    - **Expected Results:** The scanner catches the incorrect host, returns an “Invalid hostname or IP address” message, and re-prompts the user to enter a target host.
    - **Actual Result:** The scanner successfully caught the incorrect host, returned the invalid hostname message, and re-prompted the user to enter a target host cleanly.
    - **Status:** Passed

- **Evidence:**



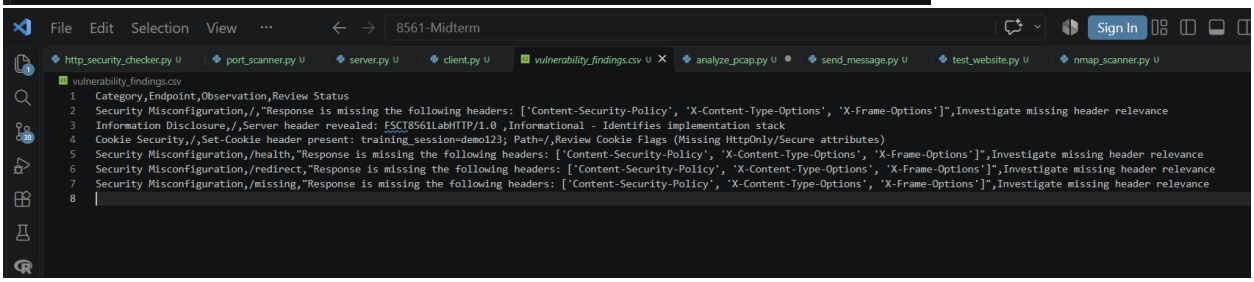
9. Running the HTTP Security Checker (success)
    - **Test Description:** Running the HTTP security checker module from the authenticated menu against the running test_website.py web server (http://127.0.0.1:8765) to scan specific endpoints (/, /health, /redirect, /missing).
    - **Expected Results:** The HTTP checker issues HTTP GET requests, inspects response headers and content types for missing security controls or version disclosures, outputs observations to the client terminal, and exports formatted results into two separate CSV files (http_scan_results.csv and vulnerability_findings.csv).
    - **Actual Result:** The HTTP checker successfully scanned all target endpoints, identified missing security headers (such as Content-Security-Policy and X-Frame-Options) and server information disclosure, logged the observations, displayed the output on the client terminal, and saved the results into the two CSV files.
    - **Status:** Passed
    - **Evidence:**

Braeden Venne



http_scan_results.csv

```
Endpoint,HTTP Method,Status,Response Length,Content Type,Server Header
/,GET,200,90,text/html; charset=utf-8,FSCT8561LabHTTP/1.0
/health,GET,200,2,text/plain,FSCT8561LabHTTP/1.0
/redirect,GET,302,0,,FSCT8561LabHTTP/1.0
/missing,GET,404,9,text/plain,FSCT8561LabHTTP/1.0
```



10. Running the HTTP Security Checker (failure)
    - **Test Description:** Running the HTTP security checker from the authenticated menu while the test_website.py web server is offline.
    - **Expected Results:** The security checker catches that the web server is offline and returns an error message to the client terminal, preventing the server from crashing.
    - **Actual Result:** The HTTP security checker successfully caught the connection refusal error and displayed the error on the client terminal without crashing and returned the client to the menu.
    - **Status:** Passed
    - **Evidence:**

```
================================================================================
|                    Security Assessment Tools                                 |
|==============================================================================|
|                                                                              |
|                    [1] Run Local TCP Port Scanner                            |
|                    [2] Run HTTP Website Security Checker                     |
|                    [3] Run Offline PCAP Analysis                             |
|                    [4] Run Message Application                               |
|                    [5] Exit Application                                      |
|                                                                              |
================================================================================

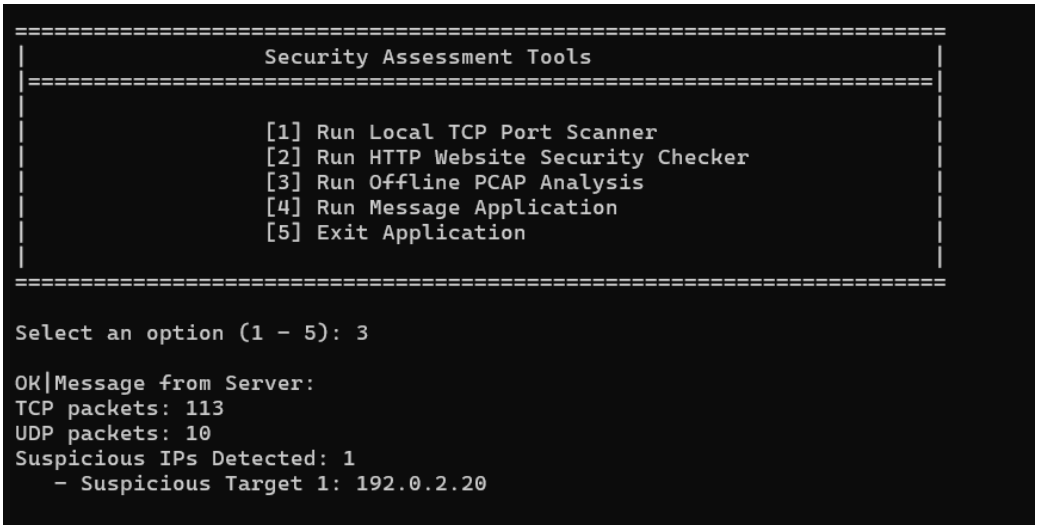
Select an option (1 - 5): 2

Starting HTTP Scan on (http://127.0.0.1:8765)...
Could not connect: HTTPConnectionPool(host='127.0.0.1', port=8765): Max retries ex
ceeded with url: / (Caused by NewConnectionError("HTTPConnection(host='127.0.0.1',
 port=8765): Failed to establish a new connection: [WinError 10061] No connection
could be made because the target machine actively refused it"))
```

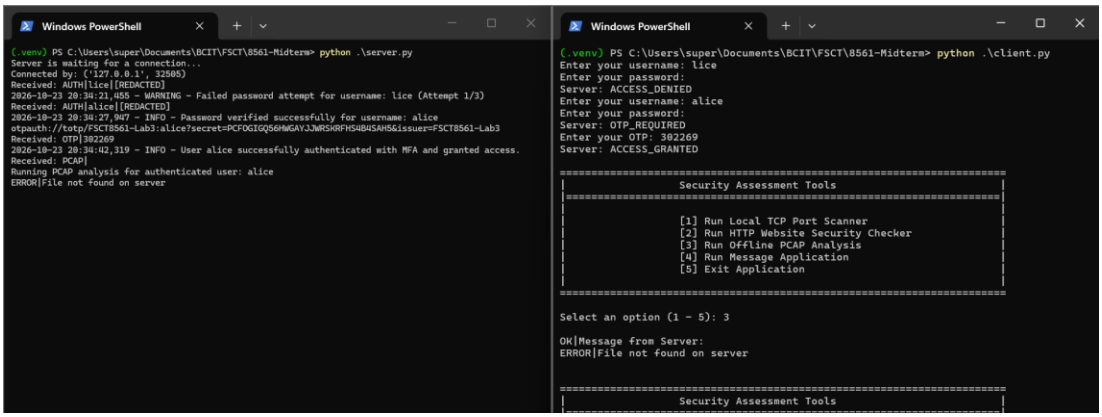
    - Tested with the test site offline

11. Running the PCAP Analysis Module (success)
    - **Test Description:** Running the PCAP analysis module from the authenticated menu to perform offline traffic analysis on the packet capture file.

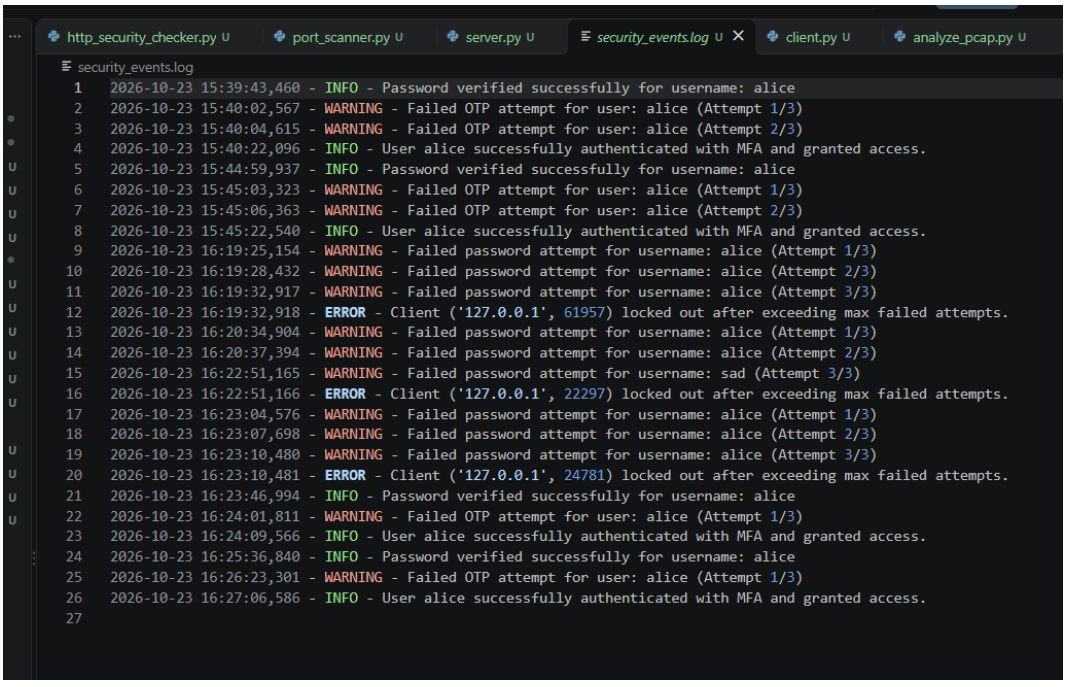
- **Expected Results:** The server verifies authentication, calls the PCAP analyzer module to parse the stored PCAP file (FSCT8561_Operation_Sentinel_Traffic.pcap) via Scapy, counts the total TCP and UDP packets, flags suspicious source Ips exceeding traffic thresholds, and returns formatted findings on the client terminal.
- **Actual Result:** The server successfully parsed the PCAP file, reported the total TCP and UDP (113 TCP and 10 UDP) packets, detected suspicious traffic from 192.0.2.20, and returned the results back to the client terminal.
- **Status:** Passed
- **Evidence:**



12. Running the PCAP Analysis Module (failure)
    - **Test Description:** Running the PCAP analysis module from the authenticated menu to perform offline traffic analysis when the target packet capture file is either missing or incorrectly named on the server backend.
    - **Expected Results:** The server catches the missing file exception during Scapy parsing, prints an error message, returns “ERROR|File not found on server” on the client menu, and prevents server crashes.
    - **Actual Result:** The server successfully caught the file error, returned the clean error message to the client terminal, and prevented crashing.
    - **Status:** Passed
    - **Evidence:**



13. Security Logging
    - **Test Description:** Verifying that critical security events, such as successful password/OTP verification, failed login attempts, account lockouts, and unauthorized command attempts are captured and recorded during operation.
    - **Expected Results:** The server should write all the security events with timestamps and log levels to “security_events.log” and the server console using Python’s logging library.
    - **Actual Result:** The server successfully tracked and recorded all critical security events to both the console and the log file.
    - **Status:** Passed
    - **Evidence:**



    - Logs are stored in a “security_events.log” file, this was done using the logging python library. Logs are also printed on the server console.

Braeden Venne

Part G – Security Analysis and Reflection

1. Which threats from your Day 1 model did your implementation mitigate, and which remain?
    - Mitigated Threats:
        - **Brute-Force Attacks:** Mitigated by setting up OTP and by having a failed login attempt lockout threshold (MAX_FAILED_ATTEMPTS = 3) that tracks both invalid password and OTP entries and terminates the socket connection.
        - **Unauthorized Port Scanning:** Mitigated by strict input validation that restricts target hosts to 127.0.0.1 (localhost).
        - **Unauthorized Command Execution / State Bypass:** Mitigated by server-side authorization checks that block commands like MSG and PCAP from being run until authentication is completed.
    - Remaining Threats:
        - **Credential Sniffing:** Remains a risk because the application communicates over unencrypted TCP sockets, which allow third parties monitoring the network channel to read plaintext credentials and messages in transit.
        - **DDoS Attack:** Remains a risk because the server is built only for a single client and doesn't have any rate-limiting or flood protection to stop someone from spamming connection requests.
2. What suspicious traffic did your PCAP analyzer identify? Cite packet-level or summary evidence.
    - The offline PCAP analyzer (analyze_pcap.py) read FSCT8561_Operation_Sentinel_Traffic.pcap and processed a total of 113 TCP packets and 10 UDP packets
    - The analyzer flagged 1 suspicious IP (192.0.2.20) by identifying any IP address that sends more than 20 packets within a 5-second window.
3. What did your HTTP checker find, and why are its findings not proof of exploitability?
    - The HTTP checker scanned endpoints (/, /health, /redirect, /missing) on http://127.0.0.1:8765 and flagged that security headers like Content-Security-Policy, X-Content-Type-Options, and X-Frame-Options were missing, the server header leaked the software stack (FSCT8561LabHTTP/1.0), and cookies lacked the HttpOnly/Secure flags
    - Automated findings should always be manually verified before confirming them as vulnerabilities. The tool relies on simple pattern matches and missing header checks, which may result in false positives or flag minor configuration settings that cannot be exploited by an attacker.
4. What are the confidentiality and replay limitations of a plain TCP connection, even with TOTP MFA?
    - **Confidentiality Limitations**: Since everything is running over unencrypted TCP, usernames, passwords, commands, and chat messages travel in cleartext. Anyone using Wireshark can just capture the traffic and read all the data.

- **Replay Limitations:** Time-based MFA stops old OTP codes from being used after their 30 second window expires, but the underlying TCP connection is still wide open. If someone captures a valid authentication stream, they can potentially hijack the active socket during that session.

5. What would you improve before deploying this application in productions
    - Transport Layer Security (TLS) should be implemented by using Python's ssl library to encrypt traffic and stop credential sniffing. Rate limiting and DDoS defenses should be implemented to prevent socket exhaustion floods.